

9주차 스터디

교재 : 자연어 처리와 컴퓨터비전

분량 : 10장

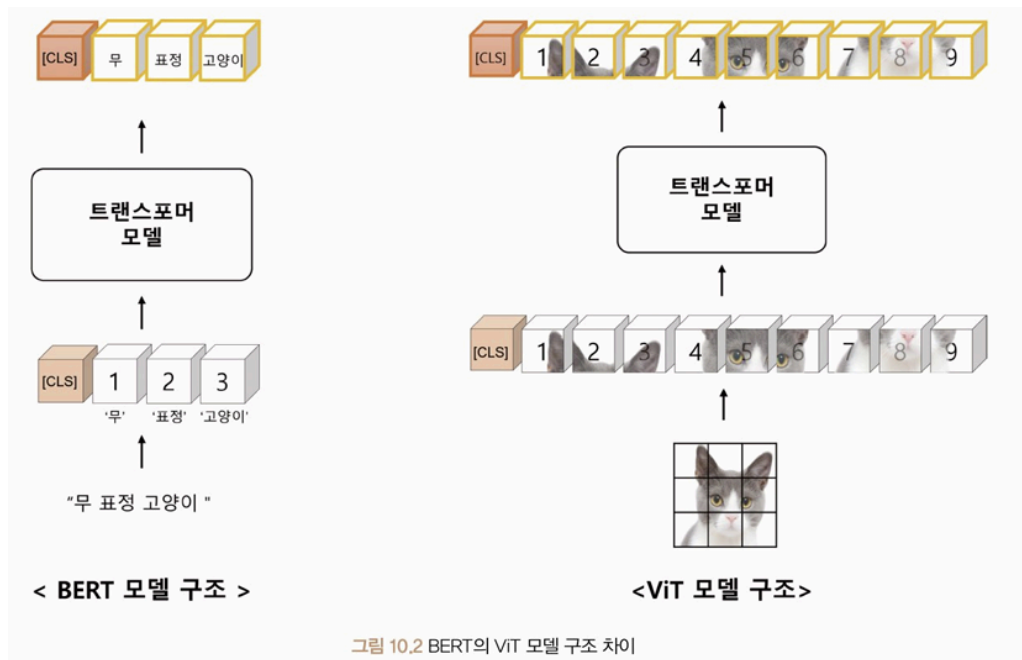
과제 기간 : ~2025-05-05

10. 비전 트랜스포머

- ViT(Vision Transformer)
 - 이미지 인식을 위한 딥러닝 모델
 - 이미지 자연어 처리 방식처럼 분류해 보려는 시도에 의해 탄생
 - 이미지 분류 모델에서 사용되는 CNN 모델의 합성곱 계층 방법을 사용하지 않는다
 - ransformer에서 사용되는 self-attention을 적용한다
 - ViT 모델은 image patch가 왼쪽에서 오른쪽, 위에서 아래 방향으로 순차적으로 입력되어 2차원 구조의 image feature를 한번에 반영할 수 없다
 - 물체의 크기, 해상도를 계층적으로 학습하는 Swin Transformer와 Convolutional Vision Transformer 모델로 위 문제를 해결한다

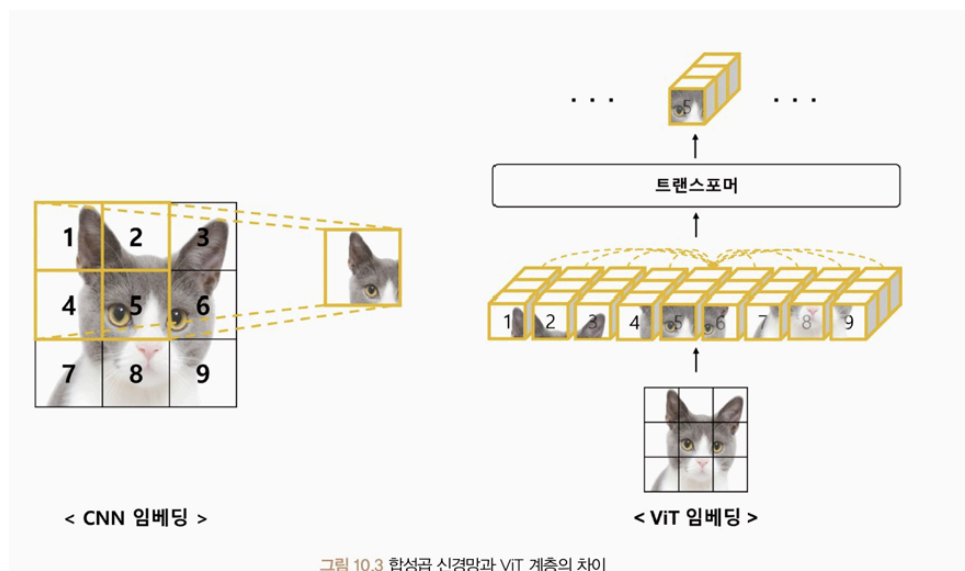
ViT(Vision Transformer)

- Transformer 구조 자체를 컴퓨터 비전 분야에 적용한 첫 번째 연구
- 기존에는 CNN에 Transformer 모델의 self-attention 모듈만 적용한 모델이 많았다
- 자연어 처리에 사용되는 BERT 모델은 문장보다 작은 단위인 Token이 순차적으로 입력된다
- ViT는 입력 이미지 패치를 왼쪽에서 오른쪽, 위에서 아래 방향으로 순차적인 배열로 가 정하여 입력한다



CNN과 ViT 비교

- 둘 다 Image Feature를 잘 표현하는 Embedding을 만드는 것이 목적
- CNN은 특정 Image Patch만 선택해 학습하고, ViT는 모든 Patch를 Self-Attention으로 학습에 관여시킨다



- Feature 추출
 - CNN: Image Patch 중 일부만 선택하여 학습하고 이를 기반으로 전체 Feature 추출

- ViT: 이미지를 작은 Patch로 나누고, 서로 간의 영향을 고려하여 전체 Feature 추출
- Layer 수
 - CNN: 좁은 Receptive Field를 가지므로 전체 Image 정보를 표현하기 위해 많은 Layer 필요
 - ViT: Attention Distance를 계산하여 하나의 Layer만으로 전체 이미지 표현 가능
- 단점
 - CNN: pixel 단위로 처리해 patch 단위보다 큰 모델이 되어 성능 저하
 - ViT: 입력 이미지 크기 고정을 위한 전처리가 필요하고, Patch의 상대적 위치 정보만 고려해 변환된 이미지에서는 Patch 간 관계가 크게 달라질 수 있음

ViT의 귀납적 편향

- 딥러닝 모델의 귀납적 편향(Inductive Bias)
 - 일반화 성능 향상을 위한 모델의 가정(Assumption)을 의미
 - 이미지 데이터는 공간적 관계를 잘 표현하므로 지역적(Local) 편향을 가진 합성곱 신경망 모델이 많이 사용됨
 - 합성곱 신경망은 이미지를 작은 조각으로 나누어 조각별 특징을 추출하고, 이를 조합하여 전체 이미지 특징을 추론
 - 지역적 특징을 강조하는 특성 덕분에 이미지 데이터에서 좋은 성능 발휘
- 시계열 데이터와 순환 신경망
 - 시계열 데이터는 시간적 관계를 잘 표현하므로 시퀀셜(Sequential) 편향을 가진 순환 신경망 모델이 많이 사용됨
 - 순환 신경망은 이전 입력 상태를 기억하고 현재 입력과 결합하여 다음 상태를 예측
 - 순차적 특성을 강조하여 시계열 데이터에서 좋은 성능을 보임
 - 그러나 특정 관계에 편향이 강할수록 다양한 관계를 표현하기 어려워 귀납적 편향이 약한 모델이 필요한 경우도 있음
- ViT 모델과 귀납적 편향
 - ViT 모델은 입력 데이터를 쿼리(Query), 키(Key), 값(Value)의 임베딩 형태로 변환하여 일반화된 관계를 학습

- 공간적 관계에 대한 강한 귀납적 편향이 없고, 다양한 관계를 학습할 수 있어 대용량 데이터에서 이미지 특징을 잘 학습
- 결과적으로 딥러닝 모델의 귀납적 편향을 줄였다

표 10.1 딥러닝 모델의 귀납적 편향

딥러닝 모델	귀납적 편향
합성곱 신경망(CNN)	지역적 편향
순환 신경망(RNN)	순차적 편향
ViT	귀납적 편향이 거의 없음

ViT 모델

- ViT 모델은 입력 이미지를 트랜스포머 구조에 맞게 일정한 크기의 패치로 나눈다
- 각 패치를 벡터 형태로 변환하는 패치 임베딩과 각 패치의 관계를 학습하는 인코더 계층으로 구성된다
- 패치 임베딩과 인코더 계층을 통해 이미지 특징을 추출하고 분류와 회귀와 같은 작업에 맞는 출력값으로 변환해 사용한다

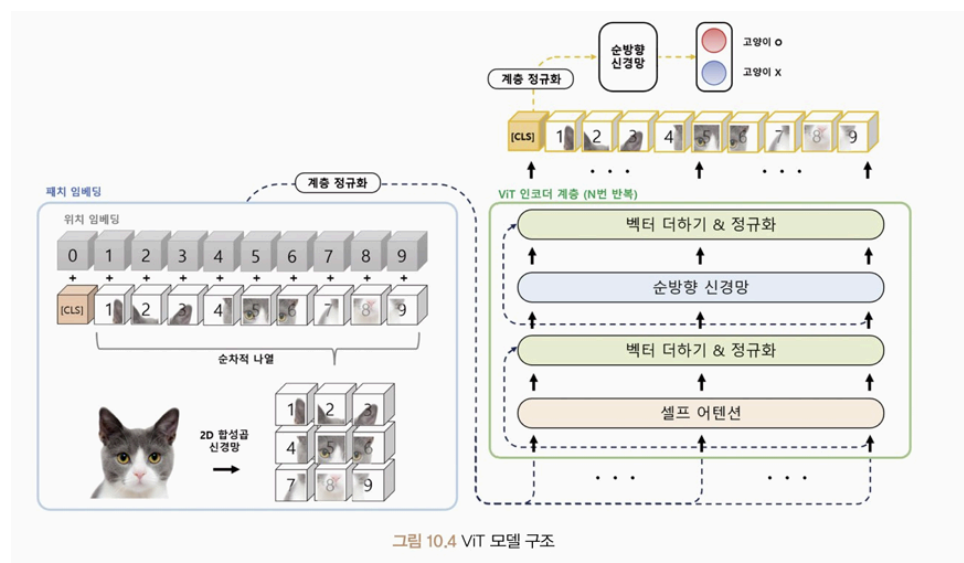
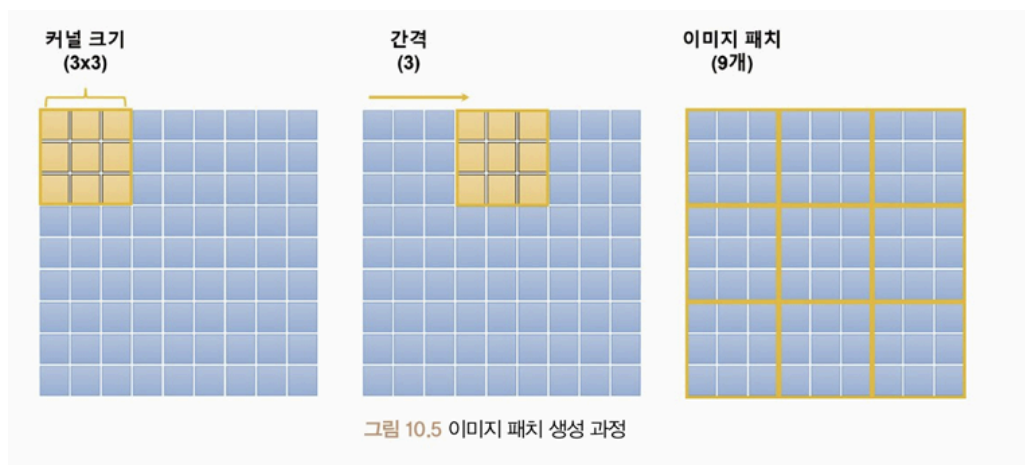


그림 10.4 ViT 모델 구조

패치 임베딩

- 패치 임베딩(Patch Embedding)

- 입력 이미지를 작은 패치로 분할하는 과정
- 입력 이미지 크기를 패치 크기에 맞추어 조정한 후, 전체 이미지를 시퀀셜 배열로 분할
- 커널 크기(Kernel Size)와 간격(Stride)을 설정하여 패치 생성



수식 10.1 패치 계산 과정

$$patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9 - 3}{3} + 1 = 3$$

- 패치 개수는 공식: $patch\ size = (image\ size - kernel\ size) / stride + 1$ 로 계산
- 분할된 패치들은 배열 형태로 나열하고, 분류를 위해 Special Classification Token(ICLS)를 추가
- 위치 임베딩(Position Embedding)
 - 패치 간 공간적 관계를 학습하기 위해 사용
 - 패치 임베딩 결과에 위치 정보를 추가하여 기존 이미지 패치 벡터들과 구분
 - 위치 임베딩을 통해 GPU 메모리 한계를 극복하고 큰 이미지를 처리할 수 있음

인코더 계층

- 인코더 계층
 - 입력된 패치 벡터를 다양한 쿼리(Query), 키(Key), 값(Value) 벡터로 변환 후 입력
 - Transformer의 Self-Attention 학습 과정과 동일하게 적용
 - 마지막 레이어에서는 분류 토큰 벡터를 이용해 최종 분류 결과 도출
 - 시퀀셜 산출물(Sequential Output) 대신 전체 이미지 특징을 표현하는 분류 토큰 벡터만 사용하여 분류 문제를 해결

Swin Transformer

- 스윈 트랜스포머(Swin Transformer) 개요
 - 2021년에 발표된 대규모 비전 인식 모델
 - 기존 트랜스포머의 고정된 패치 크기 문제를 해결하여 의미론적 분할(Semantic Segmentation) 등 작업에 적합
 - 계층적 구조를 사용하여 다양한 이미지 크기에 유연하게 대응 가능
- 계층적 구조
 - 셀프 어텐션을 이미지 크기에 따라 선형(Linearly) 복잡도를 가지도록 구성
 - 기존 트랜스포머의 2차(Quadratic) 복잡도를 해결
 - 패치를 계층적으로 처리하며 고해상도 이미지도 효과적으로 처리 가능
- 시프트 윈도우(Shifted Window) 기술
 - 입력 이미지를 일정한 크기의 패치로 분할하고 윈도우 단위로 셀프 어텐션 수행
 - 인접 윈도우 간 상호작용을 위해 윈도우를 시프트함
 - 기존 트랜스포머의 글로벌 어텐션보다 연산량 감소 및 지역 정보 강화
- 적용 가능성
 - 이미지 분류, 객체 탐지, 영상 분할 등 다양한 비전 작업에 사용
 - 트랜스포머의 구조적 이점을 계승하면서도 컴퓨터 비전에 최적화된 성능 제공

ViT와 스윈 트랜스포머 차이

- 로컬 윈도우(Local Window) 기반 처리
 - 전체 이미지를 작게 나눈 윈도우 단위로 처리하여 연산량 절감
 - 각 윈도우 내에서 어텐션을 수행하며, 계산 복잡도를 국소 영역에 한정
 - 입력 이미지 크기가 커져도 계산량이 선형적으로 증가
- 시프트 윈도우(Shifted Window) 방식
 - 인접한 윈도우 간의 상호작용을 위해 윈도우 위치를 시프트
 - 시프트된 윈도우로 다시 어텐션을 수행하여 글로벌한 문맥 정보까지 학습
 - 중첩 영역을 통해 연속성과 연결성을 확보
- 계층적 어텐션 적용
 - 로컬 윈도우와 시프트 윈도우를 번갈아 적용하여 이미지 전체를 계층적으로 인식
 - 패치 수가 증가해도 효율적으로 연산 가능하도록 구조 설계
 - 다양한 크기의 객체와 해상도에 유연하게 대응 가능
- ViT와의 차이점
 - ViT는 모든 패치에 대해 글로벌 어텐션 적용 → 높은 연산량
 - Swin Transformer는 로컬 윈도우 기반 어텐션으로 연산량 절감
 - Swin은 높은 정확도를 유지하면서도 더 넓은 범위의 이미지 크기에 적합

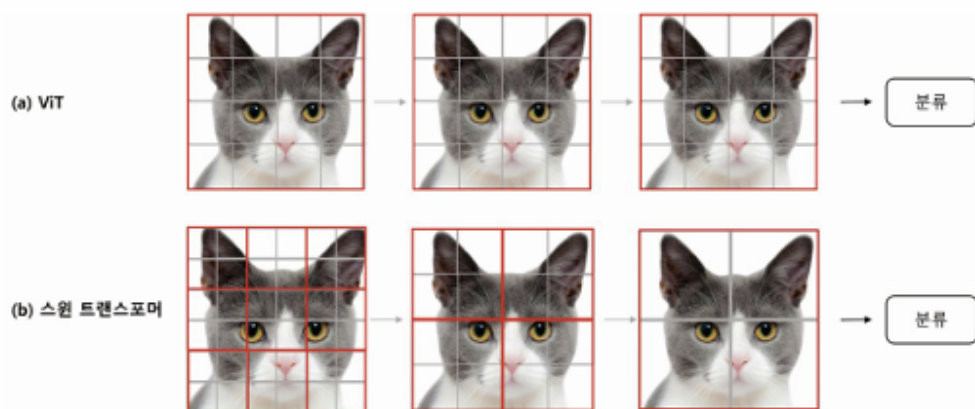


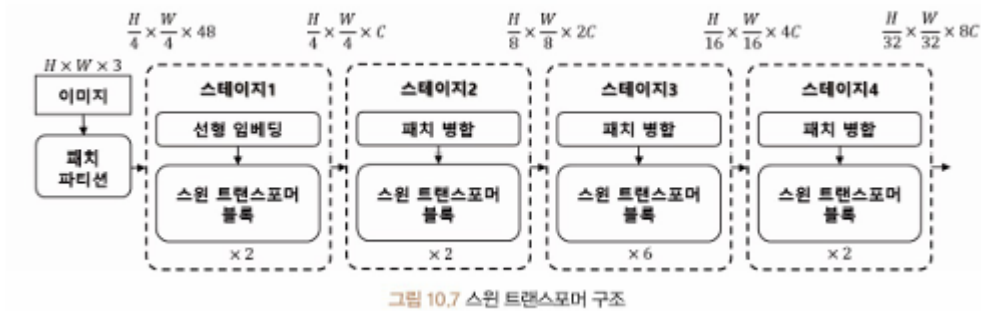
그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

표 10.4 ViT와 스윈 트랜스포머 비교

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도우 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

스윈 트랜스포머 모델 구조

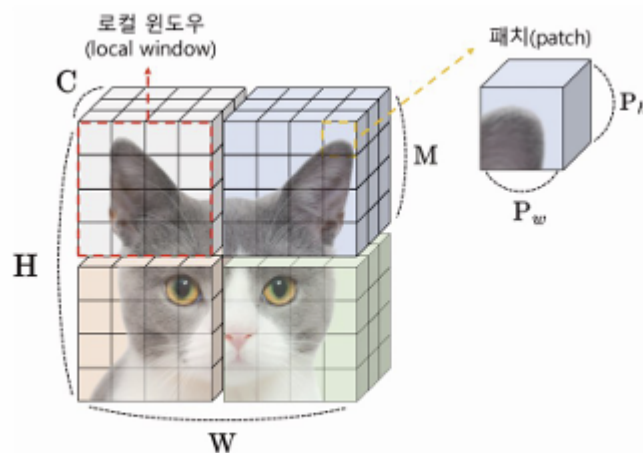
- **패치 분할 및 임베딩**
 - 입력 이미지를 일정한 크기의 패치로 분할
 - 각 패치에 대해 선형 임베딩(Linear Embedding) 수행
 - 고정된 차원의 벡터로 변환하여 모델에 입력
- **스윈 트랜스포머 블록(Swin Transformer Block)**
 - 어텐션 계층, 계층 정규화, 잔차 연결 등을 포함
 - 로컬 윈도우 기반 어텐션과 시프트 윈도우 적용
 - 동일 계층 내 패치 간 상호작용 수행
- **패치 병합(Patch Merging)**
 - 패치들을 병합하여 해상도를 줄이고 전체 이미지에 대한 분류를 점진적으로 수행
 - 병합된 패치는 순차적으로 축소되어 이미지의 전체적인 정보를 포괄
- 패치 분할 → 선형 임베딩 → 스윈 트랜스포머 블록 → 패치 병합 과정을 반복
- 계층적으로 정보 압축 및 통합하여 최종 분류 혹은 인식 작업 수행



- Swin Transformer는 총 4개의 스테이지로 구성
- 각 스테이지는 서로 다른 해상도와 패치 크기를 사용하며, 선형 임베딩 → 스윈 블록 → 패치 병합의 과정을 반복
- 1스테이지에서는 패치를 임베딩하고, 이후 스테이지에서 점차 다운샘플링 진행

• 패치 파티션 (Patch Partition)

- 입력 이미지를 작은 사각형 패치로 분할



- 분할된 패치를 트랜스포머 블록에 입력하여 공간 정보를 보존하면서 효율적으로 처리
 - $[C, H, W]$ 형태의 텐서를 $[C, H/P, W/P, P, P]$ 구조로 재구성 후 어텐션 수행
- ### • 패치 병합 (Patch Merging)
- 인접한 패치들의 정보를 결합하여 해상도를 줄이고 채널 수를 늘림

- 예: 2×2 패치를 병합하여 공간 크기를 절반으로 줄이고 채널 수는 4배로 증가
- 출력 텐서는 $[C \times 2^2, H/2, W/2]$ 형태로 재구성됨

• 스윈 트랜스포머 블록 (Swin Transformer Block)

- 각 스테이지 내에서 반복적으로 적용
- W-MSA (Window Multi-head Self-Attention)와 SW-MSA (Shifted Window Multi-head Self-Attention)로 구성
- MLP, 정규화 계층 등 포함하여 패치 간의 관계 학습

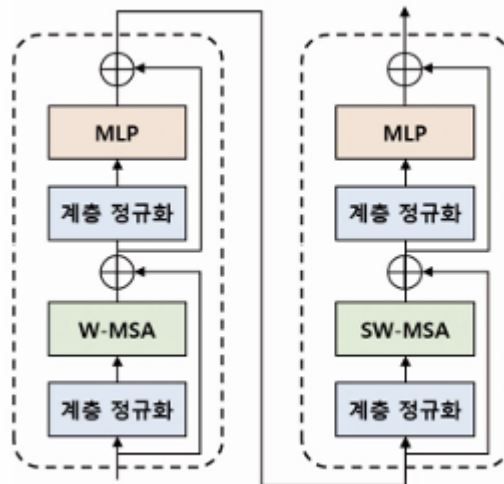


그림 10.9 스윈 트랜스포머 블록 구조

• W-MSA (Window-based Multi-head Self-Attention)

- 고정된 크기의 로컬 윈도우를 사용하여 윈도우 내에서만 다중 헤드 셀프 어텐션 수행
- 각 윈도우는 독립적으로 계산되며, 연산량 감소와 효율성 확보에 유리
- 전체 이미지가 아닌 윈도우 단위로 계산되어 계산 복잡도가 낮음 (약 1/2 감소 효과)

• SW-MSA (Shifted Window Multi-head Self-Attention)

- W-MSA의 한계를 극복하기 위한 방식
- 기존 윈도우 경계를 넘어 패치 간의 정보를 전달하기 위해 윈도우를 한 칸씩 시프트하여 어텐션 수행
- 윈도우 간 패치 관계를 학습 가능하여 더 넓은 범위의 정보 반영

- 이중 계산 구조(W-MSA → SW-MSA)를 통해 국소 정보와 전역 정보를 동시에 확보

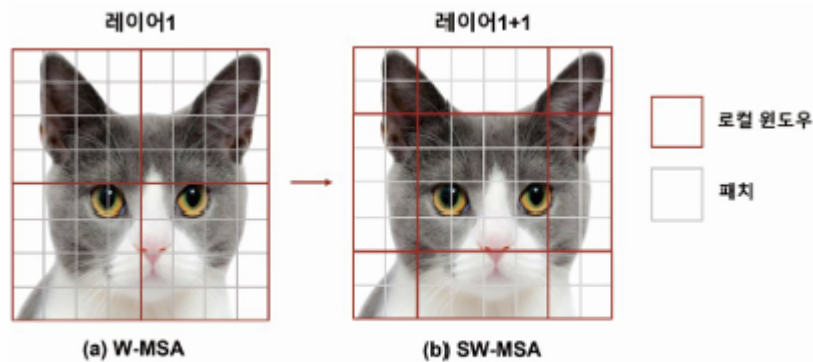


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

• W-MSA 방식

- 로컬 윈도우(빨간색 영역) 내에서만 패치 간 셀프 어텐션 연산 수행
- 윈도우 이동 없이 각각의 윈도우 내에서만 독립적으로 연산 → 연산량 증가처럼 보일 수 있음
- *효율적인 배치 계산 (Efficient Batch Computation)**을 통해 연산 병렬화 가능
- 배치 연산을 활용하면 연산 속도 빠르고 자원이 적은 작업에서도 높은 성능 확보 가능

• SW-MSA 방식의 구조적 보완

- W-MSA의 한계: 윈도우 간 상호작용이 없어 전역 정보 반영 어려움
- SW-MSA는 이를 보완하기 위해 **순환 시프트(Cyclic Shift)** 적용
- 시프트 후 로컬 윈도우를 새롭게 정의하여 윈도우 간 관계를 포함하는 어텐션 수행
- 윈도우 간 연산 간섭 방지를 위해 **어텐션 마스크 (Attention Mask)**를 병행 사용
- 결과적으로 W-MSA의 효율성과 함께 SW-MSA의 정보 연결성을 조화롭게 결합함

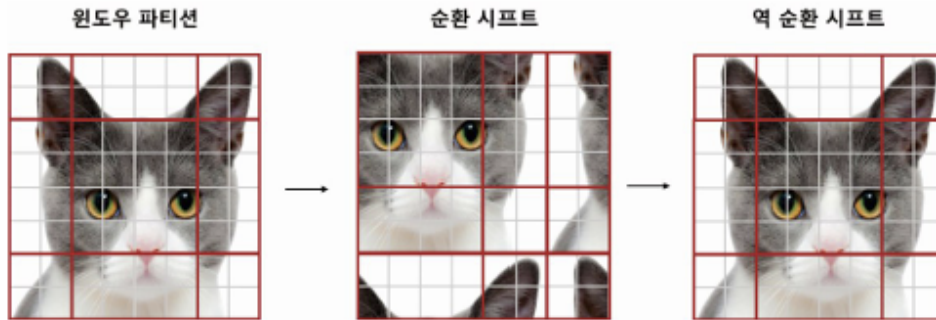


그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정

- **순환 시프트(Cyclic Shift)**

- 로컬 윈도우가 일정 크기만큼 이동하면서 **기존 위치의 정보를 새로운 위치로 옮기는 방식**
- 이동된 위치에서 셀프 어텐션 수행 시, **원래 윈도우 구조가 깨지기 때문에 어텐션 마스크가 필요**

- **어텐션 마스크(Attention Mask)**

- 연산이 ****잘못된 위치(원래 윈도우가 아닌 곳)****에서 수행되지 않도록 차단
- 즉, **서로 다른 윈도우 간의 정보 교환은 제한**하면서 연산 정확도를 보장

- **역순환 시프트(Reverse Cyclic Shift)**

- 셀프 어텐션 계산 후, 윈도우를 원래 위치로 되돌리는 작업
- 원래 윈도우 구조로 복원하여 결과 정합성 확보



그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

- **상대적 위치 편향 (Relative Position Bias)**

- Swin Transformer는 기존 ViT와 달리 상대적인 위치 관계를 고려함
- 로컬 윈도우 내의 셀 간의 상대적 거리 정보를 통해 **정확한 위치 기반 어텐션 수행**

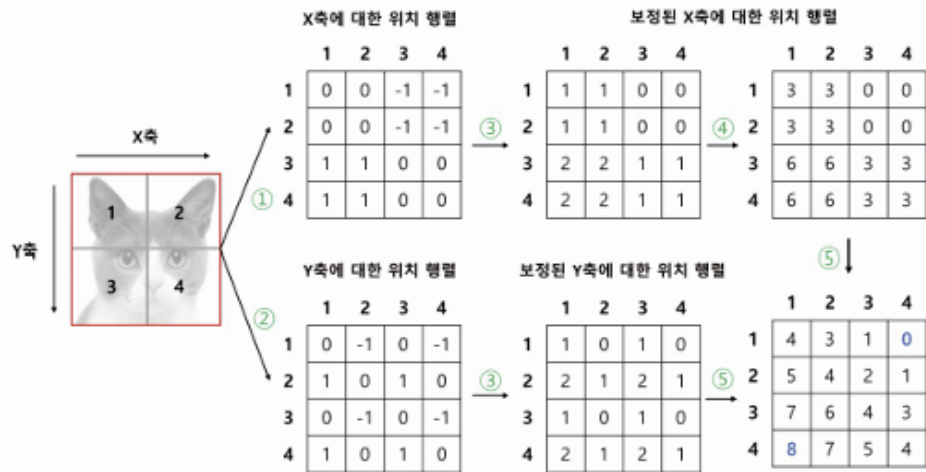


그림 10.13 상대적 위치 편향 계산 과정

CvT (Convolutional Vision Transformer)

- 합성곱 신경망 구조의 계층형 특징 추출 방식을 비전 트랜스포머(ViT)에 적용한 모델
- 계층형 아키텍처(Hierarchical Architecture)를 기반
- **계층 구조:** 다양한 크기의 로컬 윈도우를 사용하여 이미지의 지역 정보(Local Feature)와 전역 정보(Global Feature)를 동시에 학습
- **합성곱 계층 도입:** ViT와 달리 합성곱 연산을 기반으로 패치 임베딩 수행
- **계층적 토큰 추출:** 입력 이미지에서 점점 더 추상적인 특징을 계층적으로 추출

[동작 방식]

1. 입력 이미지에 대해 **합성곱 기반 패치 임베딩**을 수행
2. 각 계층에서 **로컬 윈도우 단위의 어텐션**으로 지역적 특성을 추출
3. 상위 계층으로 갈수록 **점점 더 넓은 범위의 정보를 반영**하여 전역적 이해 가능

4. 최종적으로는 전역 어텐션과 풀링을 통해 전체 이미지의 의미 파악

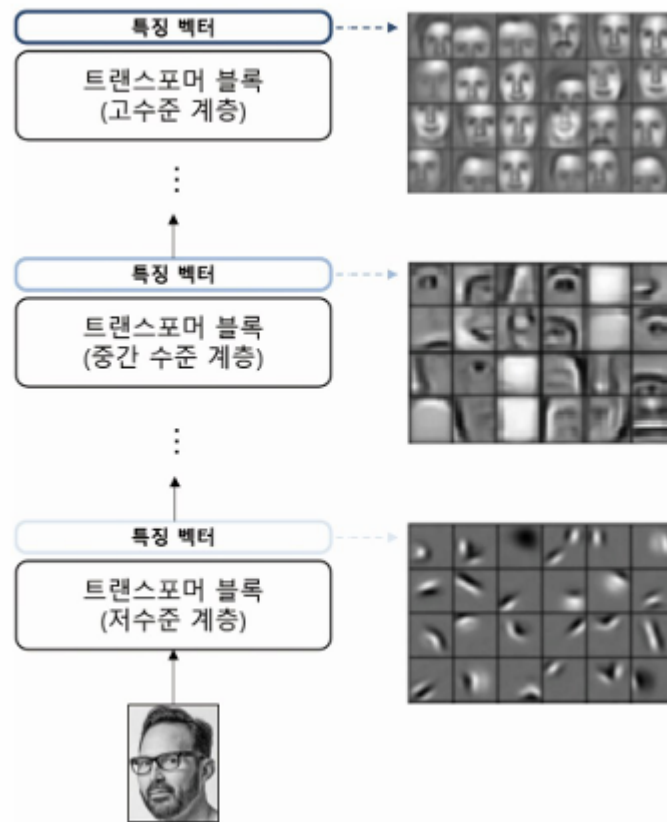


그림 10.14 CvT 모델의 계층적 특징 구조 예시

[장점]

- 기존 ViT보다 적은 계산량으로 높은 표현력
- 합성곱 기반이라 이미지의 지역 구조를 더 잘 반영함
- 계층 구조를 통해 더 적은 수의 매개변수로도 높은 성능을 보임
- ViT 대비 실제 이미지 해석에 더 유리한 구조

표 10.4 ViT와 CvT 모델 비교

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

합성곱 토큰 임베딩

- Convolutional Token Embedding
- 2D로 재구성된 토큰 맵에서 **중첩 합성곱 연산**을 통해 임베딩을 수행하는 방식

[핵심 구성 요소]

- **2D 이미지 구조 보존**
- **중첩 합성곱 연산 적용**
- 이미지 패치를 단순 펼침(flatten)이 아닌, **커널 기반의 학습 임베딩**으로 변환

[동작 방식]

1. 기존 트랜스포머 모델은 이미지 패치를 펼쳐 벡터화한 후 직접 임베딩함
2. 반면, CvT 모델에서는 이미지의 **2D 구조를 유지한 채**, 합성곱 연산을 통해 **Query, Key, Value** 임베딩 수행
3. 이렇게 얻어진 임베딩을 이용하여 **셀프 어텐션 계산과 출력 생성**

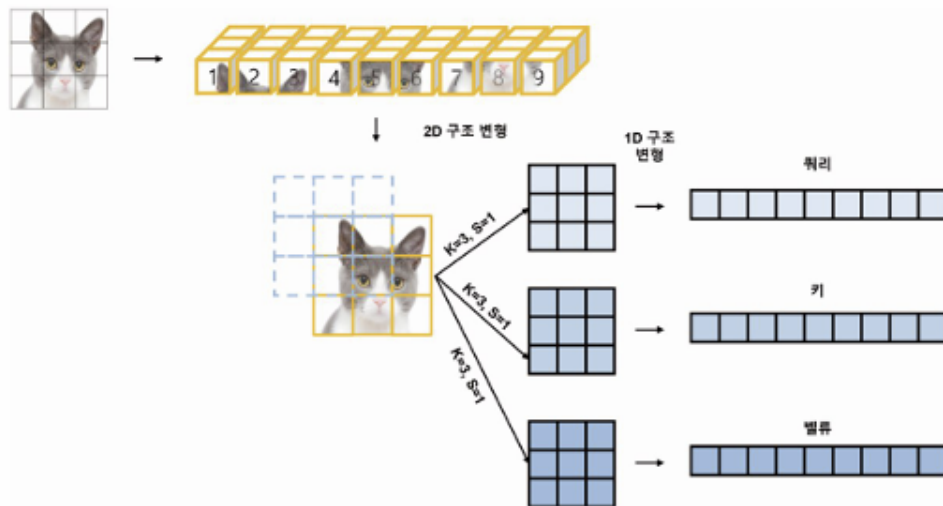


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

[장점]

- **공간 정보 보존**: 2D 구조를 유지하므로, 이미지의 공간적 관계를 더 잘 반영
- **특징 추출 강화**: 중첩 합성곱을 통해 더 풍부한 시각 특징을 추출
- **표현력 향상**: 기존 단순 선형 임베딩보다 정교한 임베딩으로 성능 향상에 기여

어텐션에 대한 합성곱 토큰 임베딩

- Convolutional Projection for Attention
- 2D로 재구성된 토큰 맵에서 **분리 가능한 합성곱 연산**을 적용하여 **성능을 유지하면서 계산 복잡도를 줄이는** 임베딩 방식

[동작 방식]

- 일반적인 Query, Key, Value 임베딩은 선형 변환으로 처리
- 여기서 CvT 모델은 각 임베딩에 대해 **작은 커널의 합성곱 연산**을 적용
- *차원 수 축소(Squeeze)**를 통해 연산량을 줄이고, 정보는 압축하여 유지

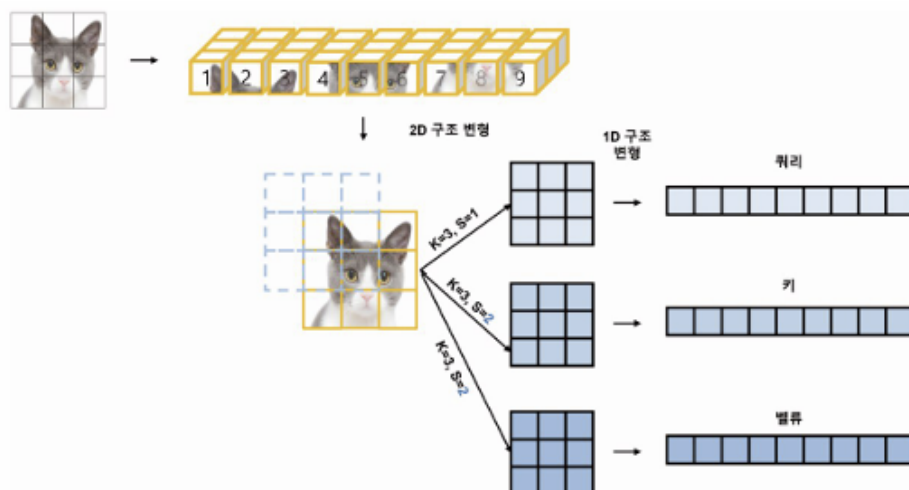


그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩

[장점]

- **계산 효율성 증가:** 연산량과 메모리 사용량을 줄여 효율적인 모델 구성 가능
- **특징 압축 유지:** 중요한 정보는 유지하면서도 모델 경량화 가능
- **성능 저하 없이 경량화 달성:** 소형 모델에도 효과적인 어텐션 구현 가능